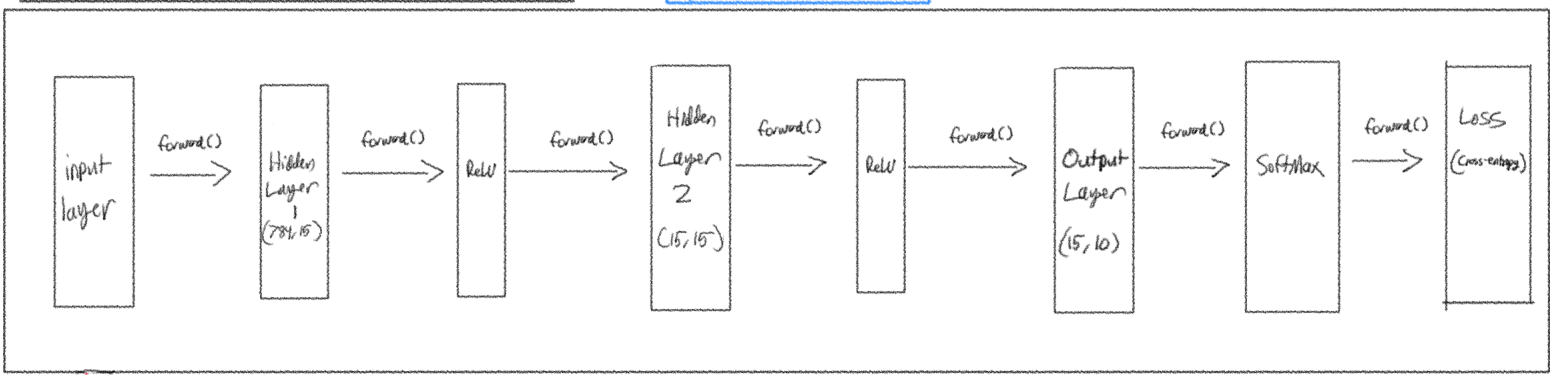
# Neural Network from Scratch:

## Forward Pass




## 1) Input:

Let $x$ be an image, $x \in \mathbb{R}^{784}$. We "flatten" each image into a vector of length 784.

So, $\vec{x}_i = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_{784} \end{bmatrix}$, where $x_i$ is a pixel in the image.

## 2.) Hidden Layer 1: `Layer_Dense(784, 15)`

$h_1 = ReLU(W_1 \vec{x}_i + b_1)$ where $W_1 \in \mathbb{R}^{15 \times 784}$, $b_1 \in \mathbb{R}^{15}$, $h_1 \in \mathbb{R}^{15}$

## 3.) Hidden Layer 2: `Layer_Dense(15, 15)`

$h_2 = ReLU(W_2 h_1 + b_2)$, where $W_2 \in \mathbb{R}^{15 \times 15}$, $b_2 \in \mathbb{R}^{15}$, $h_2 \in \mathbb{R}^{15}$

## 4.) Output Layer & Logits: `Layer_Dense(15, 10)`

$z = W_3 h_2 + b_3$, where $W_3 \in \mathbb{R}^{10 \times 15}$, $b_3 \in \mathbb{R}^{10}$, $z \in \mathbb{R}^{10}$

## 5.) Softmax: (transforms logits into probabilities)

$$p_i = \frac{e^{z_i}}{\sum_{j=1}^{10} e^{z_i}} \quad \text{or,} \quad \hat{y} = softmax(z)$$

## 6.) Cross-Entropy Loss:

Let $y$ be the one-hot encoded ground-truth vector.

$$L = -\sum_{i=1}^{10} y_i \log(p_i) = -y^T \log(\hat{p}) \quad \text{(vector form)}$$

We will show that $-y^T \log(\hat{y}) = -\sum_{i=1}^{10} y_i \log(p_i)$. Note that, $y$ is the true label vector and $\hat{y}$ is the predicted probability vector.

Proof:

Let $y = [y_1, y_2, \dots, y_{10}]^T$ and $\hat{y} = [\hat{y}_1, \hat{y}_2 \dots \hat{y}_{10}]^T$.

So, $-y^T \log(\hat{y}) = [-y_1, -y_2, \dots, y_{10}] \cdot \begin{bmatrix} \log(\hat{y}_1) \\ \log(\hat{y}_2) \\ \vdots \\ \log(\hat{y}_{10}) \end{bmatrix} = -y_1 \log(\hat{y}_1) - y_1 \log(y_2) - \dots - y_{10} \log(\hat{y}_{10})$

which is exactly, $-\sum_{i=1}^{10} y_1 \log(\hat{y}_i)$. Therefore,

$$-y^T \log(\hat{y}) = -\sum_{i=1}^{10} y_i \log(\hat{y}_i).$$

□

---

The Jacobian: The matrix of all possible partial derivatives of a multivariable vector-valued function. (A function whose domain and range are vectors)

Scalar functions have a gradient instead. Jacobian matricies show up when there are multiple outputs. (usually in the form of a vector).

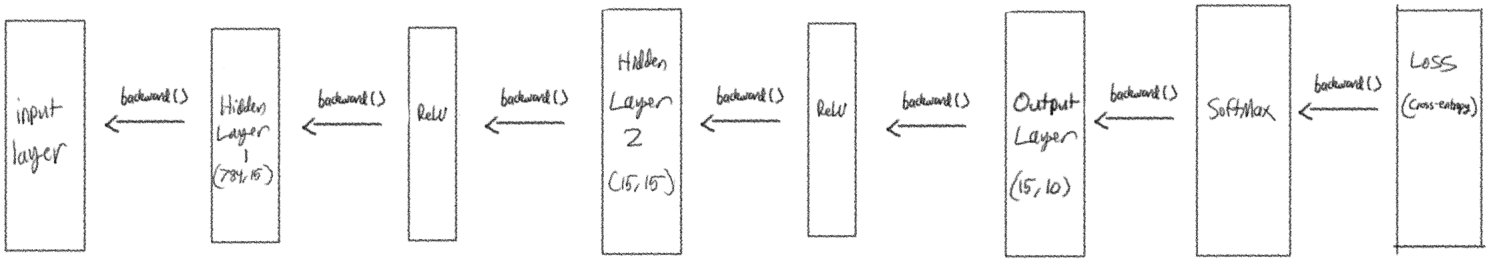
Each row in a Jacobian represents the gradient of that output with respect to each possible input.

Ex:

$$f(x_1, x_2) = \begin{bmatrix} f_1 \\ f_2 \\ f_3 \end{bmatrix} = \begin{bmatrix} x_1^2 x_2 \\ 5x_1 + \sin(x_2) \\ x_2^3 \end{bmatrix} \quad J = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} \\ \vdots & \vdots \end{bmatrix}$$

$$J = \begin{bmatrix} 2x_1x_2 & x_1^2 \\ 5 & \cos(x_2) \\ 0 & 3x_2^2 \end{bmatrix}$$

# Backpropogation:




We want to find $\nabla L$, so we utilize the chain rule with respect to all weights and biases.

So, $\nabla L = \left\langle \frac{\partial L}{\partial w_3}, \frac{\partial L}{\partial b_3}, \frac{\partial L}{\partial w_2}, \frac{\partial L}{\partial b_2}, \frac{\partial L}{\partial w_1}, \frac{\partial L}{\partial b_1} \right\rangle$

---

## 1.) Output Layer Gradient Derivation:

Starting from the final loss $L$, we compute with respect to the final logits $z$, we do this since it is standard practice due to "messiness" of trying to differentiate with respect to the softmax output.

$L = -y \log(\hat{p})$, taking the partial with respect to $z$ is denoted by

$$L = -y \log(\hat{p})$$

$$\frac{\partial L}{\partial \hat{p}} = -\frac{y}{\hat{p}} \quad \text{(using element-wise division)}$$

The Softmax function produces an $n \times n$ Jacobian matrix, since it is a vector-valued function in this context. It takes in a number of logits and outputs a vector of probabilities.

So, $\text{softmax}(\vec{x}) = [P_1, P_2, P_3, P_4, \ldots\ldots P_n]$.

So, $p_i = \frac{e^{x_i}}{\sum_k e^{x_k}}$, let $S = \sum_k e^{x_k}$. Let $j$ be the index for which variable we are differentiating with respect to. We then have two cases:

### Case 1: $i = j$

By the quotient rule we have $\frac{u' \cdot S - u \cdot S'}{S^2}$ where $u = e^{x_i}$, since $u' = \frac{\partial e^{x_i}}{\partial x_j}$ and $j = i$ we have, $u' = e^{x_i}$. $S' = \frac{\partial S}{\partial x_j}$, since with $\sum_k e^{x_k} = e^{x_1} + e^{x_2} \ldots$ the only term that would remain would be $e^{x_j}$ we have $S' = e^{x_j}$

So, $$\frac{\partial P_i}{\partial x_j} = \frac{e^{x_j} \cdot S - e^{x_j} e^{x_j}}{S^2} = \frac{e^{x_i} \cdot S}{S^2} - \frac{e^{x_i} e^{x_j}}{S^2} = \frac{e^{x_i}}{S} - \frac{e^{x_i}}{S} \cdot \frac{e^{x_i}}{S}$$

$$= \frac{e^{x_i}}{S} - \frac{e^{x_i}}{S} \cdot \frac{e^{x_i}}{S} = \frac{e^{x_i}}{S} - \left(\frac{e^{x_i}}{S}\right)^2 = P_i - P_i^2 = P_i(1 - P_i)$$

Case 2: $i \neq j$

$\frac{\partial e^{x_i}}{\partial x_j} = 0$, since it would be a constant ($i \neq j$). $\frac{\partial S}{\partial x_j} = e^{x_j}$, so for the quotient rule we have, $\frac{-e^{x_i}e^{x_j}}{S^2} = -p_i \cdot p_j$

We now merge the two cases

Let $\delta_{ij} = \begin{cases} 1 & \text{if } i=j \\ 0 & \text{if } i \neq j \end{cases}$ (A.K.A Kroderer's Delta), now we can write,

$$\boxed{\frac{\partial p_i}{\partial x_j} = p_i(\delta_{ij} - p_j)}$$

we then can represent the Jacobian with this formula,

$$J_{ij} = p_i(\delta_{ij} - p_j)$$

Now, we want to continue the chain rule with $\frac{\partial \hat{p}}{\partial z}$.

We can represent $J_{ij}$ in a vectorized form as follows,

$J_{ij} = p_i\delta_{ij} - p_i p_j$, note that $p_i\delta_{ij} = \begin{bmatrix} p_1 & 0 & \cdots & 0 \\ 0 & p_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \\ 0 & 0 & \cdots & p_n \end{bmatrix} = \text{diag}(\hat{p})$

We note also that $p_i p_j = \hat{p}\hat{p}^T$.

Thus $\frac{\partial \hat{p}}{\partial z} = J = \text{diag}(\hat{p}) - \hat{p}\hat{p}^T$.

So, $\frac{\partial L}{\partial z} = \frac{\partial L}{\partial \hat{p}} \frac{\partial \hat{p}}{\partial z} = -\frac{y}{\hat{p}}\left(\text{diag}(\hat{p}) - \hat{p}\hat{p}^T\right) = -\frac{y}{\hat{p}}\text{diag}(\hat{p}) + \frac{y}{\hat{p}}\hat{p}\hat{p}^T$

note that $\text{diag}(\hat{p}) \cdot y = \hat{p} \cdot y$ so, $-\frac{y}{\hat{p}} \cdot \hat{p} = -y$ and $\hat{p}\hat{p}^T\left(\frac{y}{\hat{p}}\right) = -p$ so we have the final formula,

$$\boxed{\frac{\partial L}{\partial z} = \hat{p} - y}$$

In a more general case we have, $\frac{\partial L}{\partial z} = (\text{diag}(p) - pp^T)g$ where $g = \frac{\partial L}{\partial p}$ for some loss function $L$. (In code, $g = dout$)

---

## 2.) Dense_Layer Gradient Derivation: (backward() for Dense_Layer)

We have for some output $y_{nm}$,

$$y_{nm} = \sum_d x_{nd} W_{dm} + b_m$$

(Deriving $\frac{\partial L}{\partial x}$):

$$\frac{\partial L}{\partial x_{nd}} = \sum_m \frac{\partial L}{\partial y_{nm}} \cdot \frac{\partial y_{nm}}{\partial x_{nd}} = \sum_m \frac{\partial L}{\partial y_{nm}} W_{dm}$$ , note that this can be represented by matrix multiplication, we take the transpose of the weight matrix and we can write.

$$\boxed{\frac{\partial L}{\partial x} = gW^T}$$ where $g = \frac{\partial L}{\partial y_{nm}}$ for some loss function $L$.

(Deriving $\frac{\partial L}{\partial W}$):

$$\frac{\partial L}{\partial W_{dm}} = \sum_n \frac{\partial L}{\partial y_{nm}} \frac{\partial y_{nm}}{\partial W_{dm}} = \sum_n g_{nm} x_{nd} = \boxed{gx^T}$$

(Deriving $\frac{\partial L}{\partial b}$):

$$\frac{\partial L}{\partial b_m} = \sum_n \frac{\partial L}{\partial y_{nm}} \cdot \frac{\partial y_{nm}}{\partial b_m} = \boxed{\sum_n g_{nm}}$$

## Gradient Update: (update())

Let $W$ and $b$ be the weight and bias matricies.

Then to update the weights we have the formula,

$$W_{new} = W - \eta \frac{\partial L}{\partial W}$$

where $\eta \in \mathbb{R}$ as the learning rate. Similar to the bias,

$$b_{new} = b - \eta \frac{\partial L}{\partial b}$$